Workload Management: DIRAC and DiracX as compute backends

Alexandre F. Boyer · CERN

12th DIRAC(X) Users' Workshop · FZU Prague · 13–16 October 2026



ARM and GPUs: HPC sees them first



276 / 500
TOP500 systems with GPUs, June 2026 (110 in 2018)

35
ARM systems in the TOP500 (5 in 2020)

Our communities
run mostly on grid sites. Some use a few HPCs opportunistically: LHCb on MareNostrum 5, up to 732 jobs at once.

What appears in HPC and clouds first reaches the grid next. The question is what it asks of the WMS.

From HPC to the grid, and to the WMS

DRIVER	HPC AND CLOUDS FIRST	GRID SITES NOW	WHAT THE WMS MUST DO
AI demand NVIDIA data-centre revenue: $194B in FY2026, 90% of its total	GPUs in 276 of the TOP500	ATLAS: ~280 GPUs at 8 grid sites, little used; no GPU pledge unit before 2028	**Schedule CPU and GPU work together; match on capabilities**
Energy and hardware cost "badly hit by AI factories" (WLCG, 2026)	ARM in 35 of the TOP500; most new AWS CPU capacity is ARM	ATLAS: ARM up to half of a pledge, 11k slots at 5 sites; Glasgow ~20% ARM	**One site, two architectures: images and matching per architecture**
More cores per node 32 per socket in 2017, 192 in 2024; memory getting scarcer	HPCs hand out whole nodes: 112 cores on MareNostrum 5	Multicore pilots: 8 cores for 48 h (CMS)	**Fill nodes, not slots: multicore payloads, bundling, memory-aware matching**

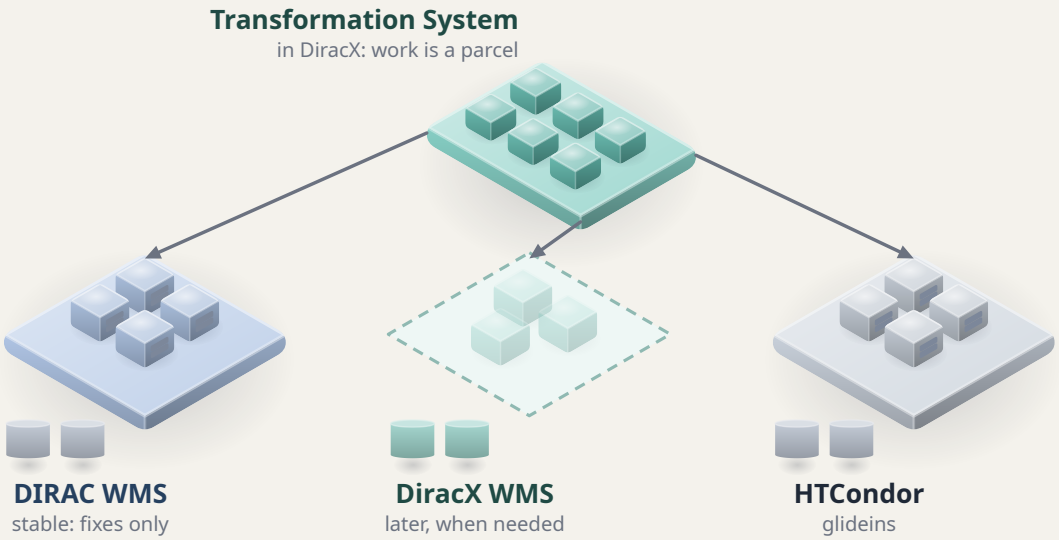
In DIRAC: temporary solutions, to keep running on these resources without changing DIRAC, and keep it stable.

In DiracX: the long-term solutions, designed for heterogeneous resources from the start.

1 · Starting from the transformation system

What it changes for workload management

The transformation system sees compute backends



each backend keeps its own data and its own data model

The transformation system (Tuesday's talks) sends parcels to compute backends. **The DIRAC WMS is one of them**, with its own data, as it is.

What separate backends give us

A stable DIRAC

The DIRAC WMS keeps its data and its behaviour. No large change: fixes and performance only, until communities move away from it.

A DiracX free of the legacy model

DiracX does not inherit DIRAC's jobs, tables or states. Work is a **parcel**; the WMS behind it is designed for it.

No rush for a DiracX WMS

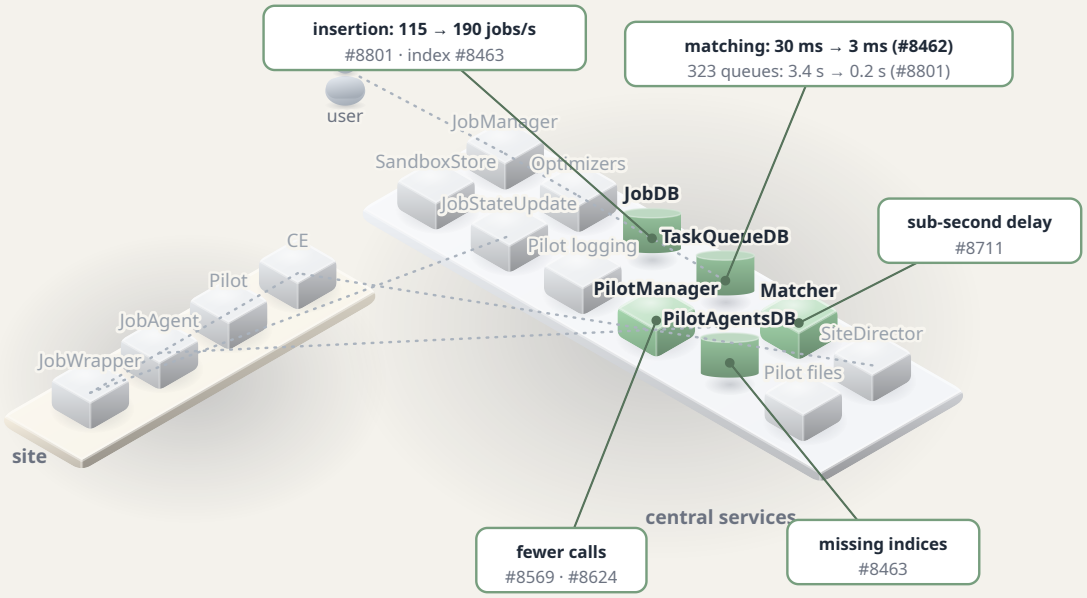
Most communities can rely on their existing WMS for now: DIRAC, or their own (see the CMS talk just before).

> The DiracX WMS is not a priority. It will move slowly, in parallel, and get a plan when the transformation system needs a native backend.

2 · DIRAC: fixes and performance

What keeps it running while we migrate

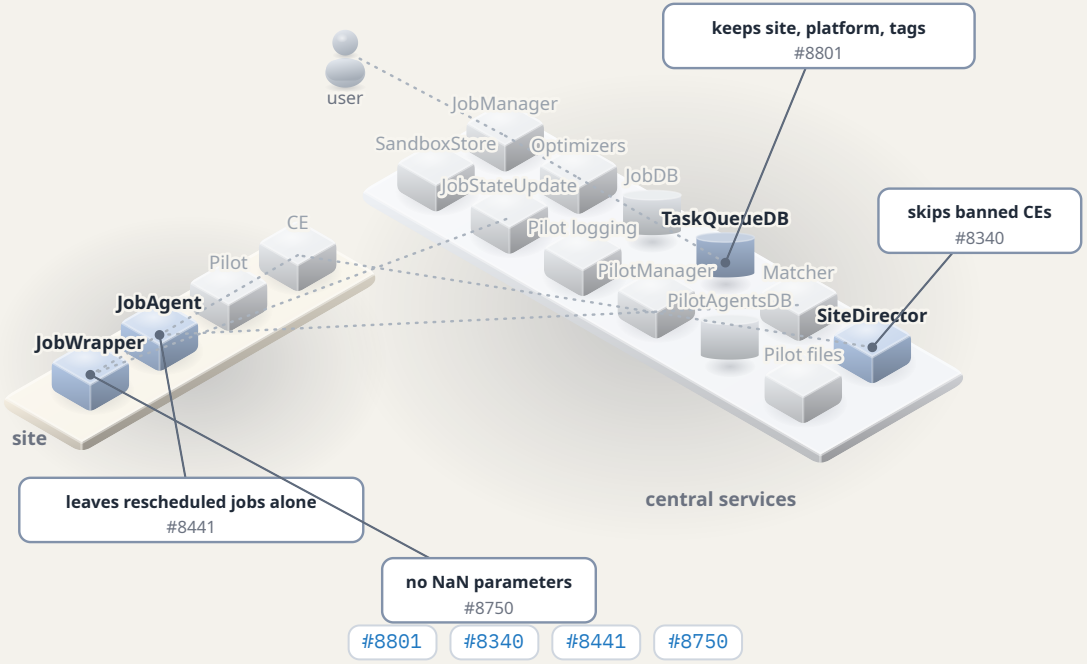
Faster



#8462 #8801 #8711 #8463 #8569 #8624 #8524

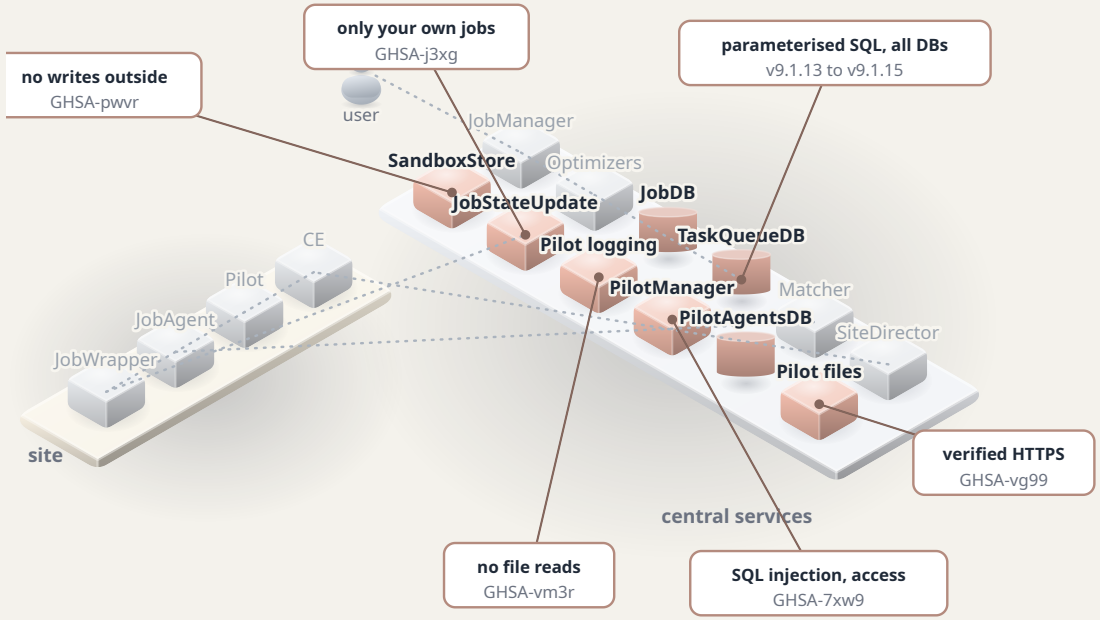
Several fixes made the DIRAC WMS faster, for installations at v9 and above. Most of these numbers need v9.1.

More reliable



> None of these change how DIRAC works. They remove ways it could fail.

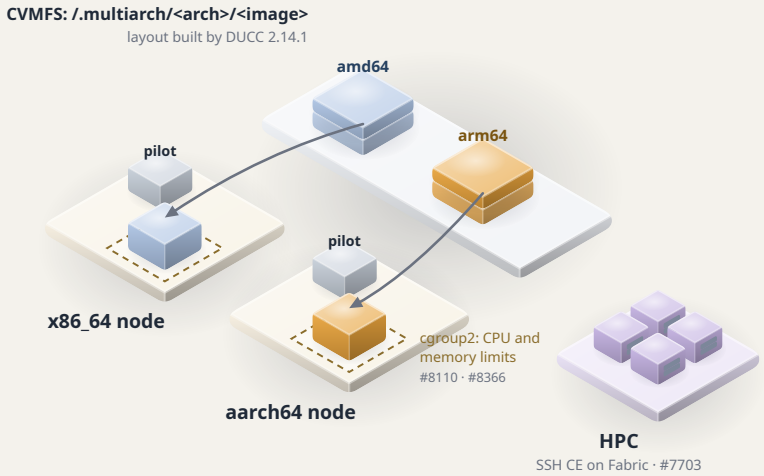
Safer



GHSA-j3xg GHSA-pwvr GHSA-vm3r fixed in v9.1.20 · v9.0.26 · v8.0.85 GHSA-7xw9 GHSA-vg99 fixed in v9.1.10 · v9.0.22

→ *DIRAC is patched, not developed:* **upgrading is how you get these**

Heterogeneous nodes



the pilot reads uname -m and picks the matching directory (#8456)

- **aarch64 images** DUW11: #8456, open, to certify. Needs CVMFS 2.14 (DUCC 2.14.1) where images are published; any client reads it.
- **Limits** DUW11 and **HPC** DUW11: in v9.0.
- **Not solved in DIRAC**: a job for x86_64 *or* aarch64.

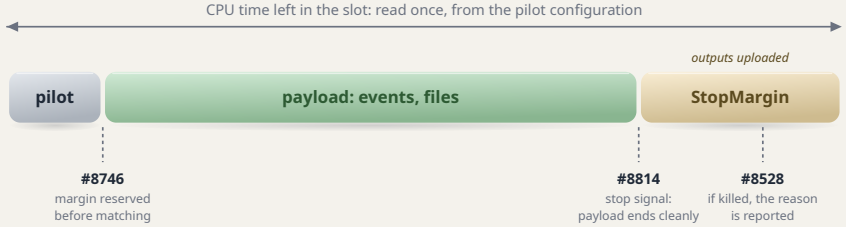
Time left in the slot

The problem

- Jobs in containers could not ask the batch system for the time left.
- The post-processing margin was known only to the Watchdog, which could only kill.

An example: an MC job sized for 183 events produced 150, was killed, and its output was never uploaded.

Since v9.1.4 (#8479): the pilot reads the time left once; jobs read it from the pilot configuration.

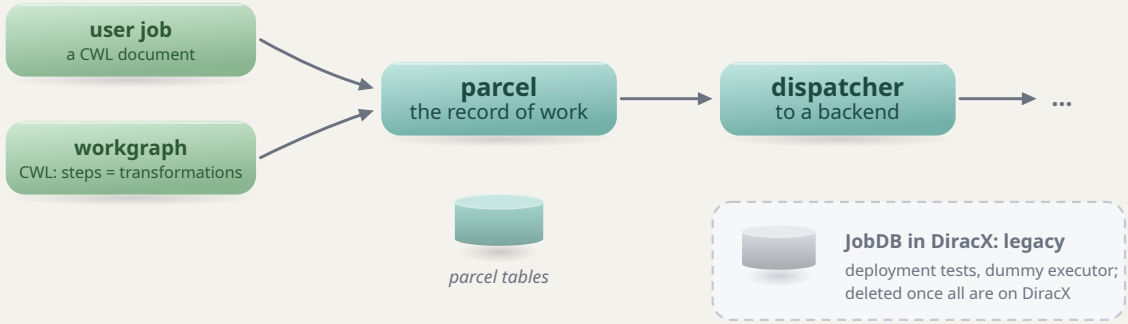


> The three fixes above are in **v9.1.21** only; see "Allocated time management" in the Administrator Guide. CPU work is normalised with DB12: next talk.

3 · DiracX: parcels, not jobs

And the proofs of concept around the future WMS

No jobs in DiracX



- A user job is a parcel, like a bundle of transformation inputs. A failed parcel is final: resubmitting creates a new one.
- We do not port job monitoring or the job manager DUW11.

- The JobDB in DiracX is legacy. It receives job status updates from DIRAC, to **test that a DiracX deployment works**.
- It also feeds the **dummy job executor** (diracx#995): jobs go `Waiting → Matched → Running → Done` without running anything. A stand-in backend, to test the transformation system end to end.
- **Once everyone is on DiracX, it is deleted.**

CWL describes all the work

One format (DX-ADR-007, draft): a user job, a workgraph and each transformation are CWL v1.2 documents.

- DiracX settings are **hints** under `dirac:`, never requirements: every document stays plain, runnable CWL.
- The same document runs with `cwltool`, locally against grid data, and at scale.
- Processes are resolved before dispatch: the node runs without fetching definitions.

What this means for a WMS backend

- Matching requirements come from the hints.
- The job wrapper resolves inputs (`lfn:`, `sandbox:`), captures the output sandbox, and writes a status report per input.
- Outputs are registered by DiracX after the parcel ends, not by the wrapper.

Today: `dirac-cwl` 1.3.0; an experimental CWL job wrapper in DIRAC (#8425). The CWL job submission prototype for DiracX DUW11 became part of the transformation system.

interCEde: one library for computing elements

A typed, asynchronous library to submit to and manage computing elements, usable by DIRAC, DiracX and other workload managers. github.com/DIRACGrid/intercede, proof of concept

Design

Three ADRs under review (intercede #4): computing elements, integration tests, credentials.

Open points: per-site submit options, whole-node and GPU requests, ARC delegation renewal, cleaning up cloud VMs.

Code

Core contract proposed this week (#27).

Integration-test scaffolding (#25), and test setups from the communities: HTCondor, ARC with Slurm, Slurm.

Next

Merge the ADRs, then: core, registry, integration tests, backends.

Last, a synchronous facade and an adapter so DIRAC can use it (#24).

In the backend model, a WMS backend owns the policy (matching, pilots); interCEde owns the mechanics of talking to a CE.

A matcher for the DiracX WMS

Goal (diracx #843): match pilots to work at **1 kHz** (200 Hz at least), with 10 million waiting items per VO. Redis holds an index that can be rebuilt from the database.

Built: matching with tag expressions, prioritisation, a benchmark harness, a Redis data model (release 0.4.0, Sep 2026).

Measured: the pure Python baseline only, 210 ms per match at 50k candidates, 5.7 s at 750k. No Redis numbers yet.

What we learned

- Real requirements vary little: LHCb has 872k waiting jobs but 14 distinct (CPU time, group) pairs. Group work by profile, as task queues do; do not test items one by one.
- An item eligible at ~49 sites must not be stored 49 times.
- Work runnable on two architectures needs a *list* of requirements.

Next: matching inside Redis (Lua) and its benchmark on realistic data (#873), then scale tests (#874). **Then an ADR**, written from those numbers, for the matcher of the DiracX WMS backend.

Pilots: the re-thinking has not started

Landed

- A pilot database and router in DiracX (#570, Aug 2026) DUW11.
- Pilot: Python 2 removed DUW11; maintenance fixes.

Paused

- Pilot registration and authentication with pilot tokens (#421).
- Pilot logging through DiracX (#550).
- Untouched since mid-2025.

Questions for later

- What does a pilot need to know about parcels?
- What belongs to the pilot, what to the job wrapper?
- One model for pull and push?

> Pilots belong to the DiracX WMS backend, which comes after the transformation system. We continue to patch the DIRAC pilot.

The job wrapper: one for all of us, owners wanted

Today there are several

- DIRAC's JobWrapper: pre-processing, payload, post-processing; an offline variant for push.
- The `dirac-cwl` wrapper, experimental in DIRAC.
- Executor prototypes in DiracX.

Each community also adds its own steps around the payload.

Requirements to gather

- Inputs: data, sandboxes, protocol access.
- Time left, graceful stop, containers, architectures.
- Nodes without outbound network.
- Reporting: status, parameters, logs.
- What happens when an upload fails.

We are looking for owners: people who collect the requirements from the communities, set priorities, and accept the result, in a product-owner role. Come and talk to us this week.

Take-aways

THE MODEL

- The transformation system sends parcels to compute backends; the DIRAC WMS is one of them.
- DIRAC stays stable; DiracX is free of DIRAC's job model.

FOR INSTALLATIONS

- Upgrade to v9.1: faster matching and job insertion, time-left fixes (v9.1.21), security patches.
- Multi-architecture images: #8456, soon.

FOR DIRACX

- No jobs: parcels. The JobDB in DiracX only tests deployments, then goes.
- The DiracX WMS is not a priority; it will get a plan when needed.
- CWL describes everything that runs.
- Matcher: Redis numbers, then an ADR. interCEde: ADRs close to merge. Pilots: not started.

WE NEED YOU

- Owners for the job wrapper.

Thank you

Questions and comments welcome, this week and after.

Alexandre F. Boyer · alexandre.boyer@cern.ch

github.com/DIRACGrid · diracx.diracgrid.org

